

STUDENT NAME : SAFREENA BANU S

PROJECT TITLE : SALES FORECASTING USING HISTORICAL DATA

1. PROBLEM STATEMENT

Accurate sales forecasting is challenging due to changing demand, seasonality, and market conditions. Although businesses have large amounts of historical sales data, traditional forecasting methods often fail to capture sales trends and seasonal patterns, resulting in inaccurate predictions. This project analyzes historical monthly sales data from 2019 to 2025 using time-series analysis techniques to build a reliable forecasting model. The model predicts future sales trends, helping businesses improve inventory planning, demand management, and decision-making.

2. ABSTARCT

Sales forecasting is an important task for effective business planning and decision-making. This project focuses on forecasting future sales by analyzing historical sales data collected from 2019 to 2025. The dataset is first preprocessed and aggregated on a monthly basis to understand long-term trends and seasonal patterns in sales.

Exploratory Data Analysis (EDA) is performed to identify trends, seasonality, and variations in the sales data. To accurately model the time-series behavior of sales, a Seasonal ARIMA (SARIMAX) model is applied. Stationarity of the data is verified using statistical tests, and appropriate model parameters are selected to capture both trend and seasonal components.

The trained SARIMAX model is used to forecast sales for future months, and the predicted values are compared with historical data for validation. The results demonstrate that time-series forecasting using historical data can provide reliable and meaningful sales predictions. This approach helps businesses improve demand forecasting, inventory management, and strategic planning.

3. SYSTEM REQUIREMENTS

- **Hardware:**

- **Processor:** Intel Core i3
- **RAM:** Minimum 4 GB (8 GB recommended)
- **Hard Disk:** Minimum 100 GB free space
- **System Type:** 64-bit machine
- **Display:** Standard monitor with minimum 1366×768 resolution

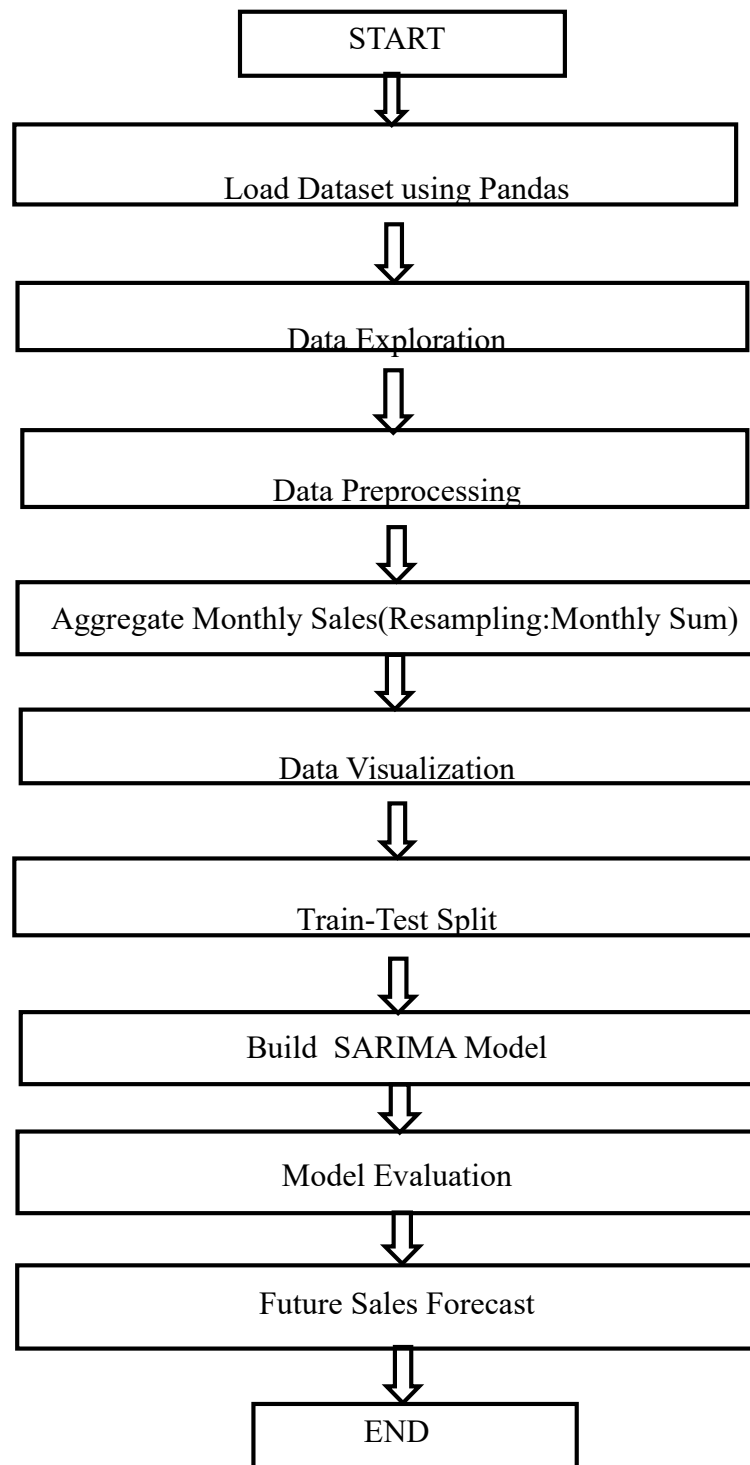
- **Software:**
 - **Operating System:** Windows 11
 - **Programming Language:** Python
 - **Integrated Development Environment(IDE):** Jupyter Notebook
 - **Libraries & Frameworks:**
 - NumPy
 - Pandas
 - Matplotlib

4. OBJECTIVES

The primary objective of this project is to analyze historical sales data in detail to identify meaningful trends, seasonal variations, and recurring patterns that influence sales performance over time. To ensure reliable and accurate analysis, the collected sales data is thoroughly preprocessed and cleaned by handling missing values, removing inconsistencies, and transforming the data into a suitable format for time-series analysis. Based on the processed data, a time-series forecasting model is developed to predict future sales by learning from past sales behavior and seasonal fluctuations.

The project focuses on generating accurate and reliable sales forecasts that can assist organizations in effective business planning and strategic decision-making. Furthermore, the forecasting results provide valuable predictive insights that help improve inventory planning, demand management, and resource allocation, thereby reducing stock shortages, minimizing excess inventory, and enhancing overall operational efficiency.

5. FLOWCHART OF THE PROJECT WORKFLOW



6. DATASET DESCRIPTION

- **Source** : The dataset used in this project is obtained from **Kaggle**.
- **Type** : Public dataset
- **Time Period** : Sales data from 2019 to 2025
- **Size** 3000 rows x 15 columns
- **Dataset Attributes** :

Column Name	Description
Date	Date of the sales transaction.
Year	Year in which the sale occurred.
Month	Month of the sales transaction.
Product_ID	Unique identifier for each product.
Product_Name	Name of the product.
Category	Product category (e.g., Electronics, Accessories).
Store_ID	Unique identifier for each store.
City	City where the store is located.
Units_Sold	Number of units sold.
Sales	Total sales count for the transaction.
Revenue	Total revenue generated from sales.
Discount	Discount percentage applied.
Promotion	Indicates whether promotion was applied (Yes/No).
Stock_Level	Available stock level after sales.
Profit	Profit earned from the transaction.

[3]:

	Date	Year	Month	Product_ID	Product_Name	Category	Store_ID	City	Units_Sold	Sales	Revenue	Discount	Promotion	Stock_Level	Profit
0	2020-05-29	2020	May	P103	Headphones	Accessories	S02	Madurai	8	8	20400	15	Yes	13	6038
1	2024-10-16	2024	October	P105	Camera	Electronics	S03	Coimbatore	17	17	361250	15	Yes	78	81928
2	2023-06-30	2023	June	P105	Camera	Electronics	S04	Bangalore	9	9	191250	15	No	18	21144
3	2020-01-18	2020	January	P102	Laptop	Electronics	S02	Madurai	9	9	405000	10	Yes	29	48322
4	2025-02-15	2025	February	P104	Smartwatch	Electronics	S03	Coimbatore	9	9	85500	5	Yes	99	22130

7. DATA PREPROCESSING

- **Missing Values:**

Dataset was checked for missing values using `isnull()` and no missing values were detected in the sales data.

- **Duplicates:**

Duplicate records were verified using `duplicated()` and no duplicate rows were found.

- **Outliers:**

- Outliers were identified using box plots and Interquartile Range (IQR) method on monthly aggregated sales data.
- Detected outliers were not removed, as they represent real-world business scenarios such as seasonal demand, festivals, or promotional sales spikes.

- **Encoding:**

- **Not applied**, since SARIMA is a **time-series forecasting model** that works only on a **single numerical variable (Sales)**.
- No categorical features were used for model training.

- **Scaling:**

- **Not required** for SARIMA, as it is a statistical time-series model based on historical patterns and does not depend on feature distance.
- Sales values were used in their original scale to preserve trend and seasonality.

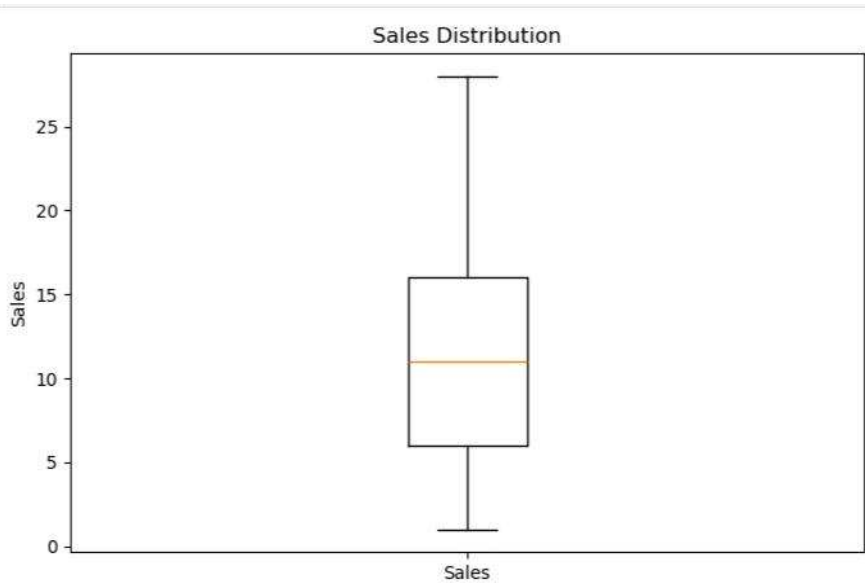
8. EXPLORATORY DATA ANALYSIS (EDA)

- **Univariate Analysis - Box Plot (Sales Distribution) :**

- Middle line (inside box) → Median sales
- Box (Q1 to Q3) → Middle 50% of sales values
- Whiskers → Lowest and highest sales values
- Outliers (if any) → Unusually high or low sales

Key insights:

- Sales values are moderately spread
- No extreme outliers → data is stable
- Useful to understand sales variability



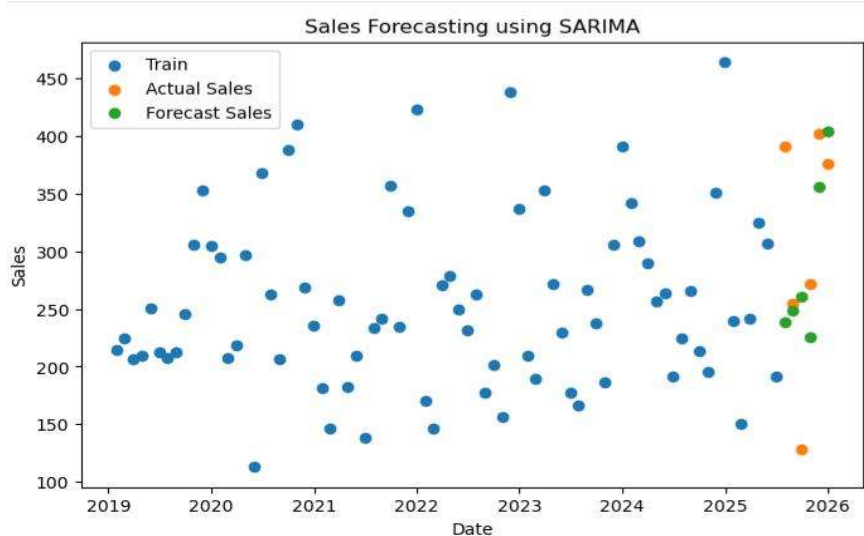
- **Bivariate / Multivariate Analysis:**

- **Scatter Plot:** Comparison between,
 - Training data (Blue dots)
 - Actual sales (Orange dots)
 - Forecasted sales (Green dots)

Dots represent individual monthly sales values

Key insights:

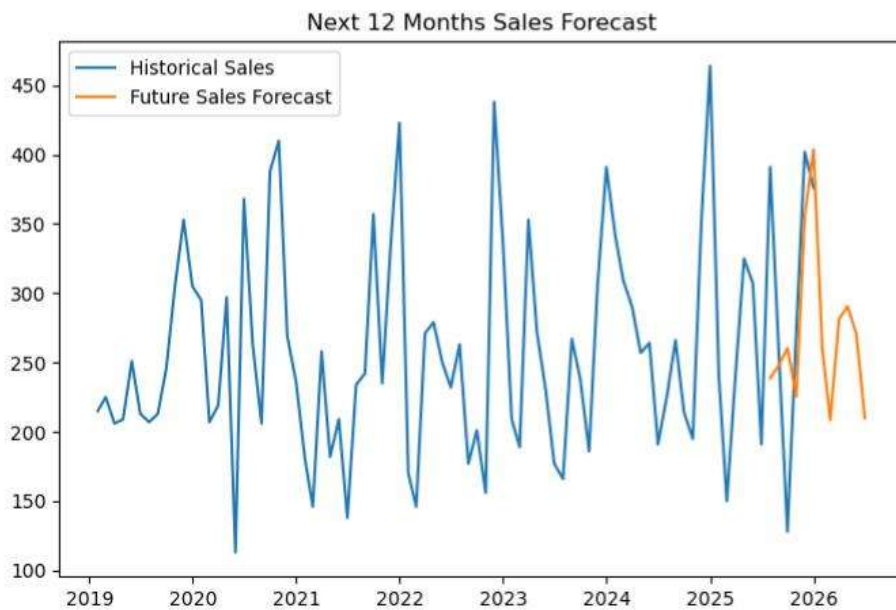
- Forecasted points are close to actual sales.
- Shows SARIMA model is reasonably accurate.
- Helps evaluate model performance.



- **Line Plot (Time Series Line Chart)**
 - Blue line: Historical sales data
 - Orange line: Future sales forecast (next 12 months)

Key insights:

- Continuous line shows trend over time
- Forecast line continues from historical data



9. FEATURE ENGINEERING

- **New Features:**
 - **Monthly Aggregation:**
Daily sales data was aggregated into monthly total sales using time-based resampling to capture long-term trends and seasonality.
 - **Lag Features:**
Previous time period sales values (lag features) were implicitly used by the SARIMA model to learn historical dependencies.
 - **Seasonality Components:**
Monthly seasonality (12-month cycle) was incorporated through the seasonal order (P, D, Q, 12) in the SARIMA model.

- **Feature Selection:**
 - **Univariate Feature Selection:**
Only the Sales variable was selected for forecasting, as SARIMA is a univariate time-series model.
 - **Removed Redundant Variables:**
Non-time-dependent columns such as Product ID, Store ID, and City were excluded to avoid noise and maintain temporal consistency.
- **Impact:**
 - Improved forecasting accuracy by focusing on historical sales patterns and seasonality.
 - Reduced model complexity by eliminating irrelevant features.
 - Enabled better trend and seasonal pattern detection in long-term sales forecasting.

10. MODEL BUILDING

- **Model Tried:**
SARIMA (Seasonal AutoRegressive Integrated Moving Average)
- **Why these Models:**
 - SARIMA is well-suited for time-series sales data as it effectively captures:
 - Trend
 - Seasonality
 - Autocorrelation in historical sales patterns.
 - It performs better than basic regression models when sales data shows repeating seasonal behavior.
- **Training Details**
 - Historical monthly sales data was converted into a time-series format with the date as index.
 - The dataset was split based on time order:
 - 80% Training data
 - 20% Testing data
 - SARIMA model parameters $(p,d,q)(p,d,q)(p,d,q)$ and seasonal parameters $(P,D,Q,s)(P, D, Q, s)(P,D,Q,s)$ were selected based on data characteristics and model performance.
 - The model was trained using the training data and evaluated on the test data.

11. MODEL EVALUATION

The performance of the **SARIMA (Seasonal AutoRegressive Integrated Moving Average)** model was evaluated using standard regression error metrics and visual inspection techniques.

- **Residual Analysis**

- Residuals were calculated as the difference between actual sales and predicted sales.
- Residual values were randomly distributed around zero.
- No strong trend or systematic pattern was observed, indicating that the model fits the data well and No major bias or seasonality remained in the residuals.

- **Visual Evaluation**

The following plots were used to visually evaluate model performance:

- Actual vs Forecast Sales Plot
- Train, Test, and Forecast Comparison Plot

Metric	SARIMA Model
MAE	68.64837369178852
RMSE	87.4116376843469

Lower MAE and RMSE values indicate better forecasting accuracy.

12. DEPLOYMENT

Deployment Method: Jupyter Notebook–deployment(used for learning, testing, and academic projects)

Public Link: <https://jupyter.org/try-jupyter/lab/>

UI Screenshot:

A screenshot of a Jupyter Notebook interface. It shows a code cell with the following text: 'In [28]: sales_forecast=model_fit.forecast(steps=6)' followed by 'sales_forecast.index=test.index' on the next line. The code is highlighted in a light blue background. Above the code cell, there are several small icons for notebook navigation (back, forward, home, etc.).

```
In [28]: sales_forecast=model_fit.forecast(steps=6)
sales_forecast.index=test.index
```

Sample Prediction:

The model predicts sales for the next 6 months (or next 12 months) such as January 2026 to June 2026 using lpast sales trends and seasonality. In the implemented code,

```
Sales_forecast=model_fit.forecast(steps=6)
```

```
Sales_forecast.index=test.index
```

Predicts sales for the next 6 months (for example, from January 2026 to June 2026).

```
Sales_forecast=model_fit.forecast(steps=12)
```

```
Sales_forecast.index=test.index
```

Predicts sales for the next 12 months.

13. SOURCE CODE

#Load the Dataset

```
import pandas as pd
```

#Read the Dataset

```
df=pd.read_csv("sales_forecast_2019_2025.csv")
```

#Data Exploration

#Display first few rows

```
df.head()
```

#Shape of the Dataset

```
print("Shape",df.shape)
```

#Column names

```
print("Columns:",df.columns.tolist())
```

#Data types and non-null values

```
df.info()
```

#Summary statistics for numeric features

```
df.describe()
```

#Check for Missing Values and Duplicates

#Check for Missing Values

```
print("Missing Values:\n",df.isnull().sum())
```

#Check for Duplicates

```
print("Duplicate values:",df.duplicated().sum())
```

#Data Preprocessing

```
df['Date']=pd.to_datetime(df['Date'])
```

```
df=df.sort_values('Date')
```

```
df.set_index('Date',inplace=True)
```

#Aggregate Monthly Sales(Historical Data)

```
monthly_sales=df['Sales'].resample('ME').sum()
```

```
monthly_sales.head()
```

#Visualize a Few Features

```
import matplotlib.pyplot as plt
```

#Visualise Sales Trend

```
Plt.figure(figsize=(8,5))
```

```
Plt.boxplot(df['Sales'],tick_labels=['Sales'])
```

```
Plt.title("Sales Distribution")
```

```
Plt.ylabel("Sales")
```

```
Plt.show()
```

#Outliers Detection

```
Q1=df['Sales'].quantile(0.25)
```

```
Q3=df['Sales'].quantile(0.75)
```

```
IQR=Q3-Q1
```

```
Lower_bound=Q1-1.5*IQR
```

```
Upper_bound=Q3+1.5*IQR
```

```
Outliers=df[(df['Sales']<lower_bound)|(df['Sales']>upper_bound)]
```

```
Print(outliers)
```

```
Print("Number of outliers:",outliers.shape[0])
```

#Stationary Test

```
from statsmodels.tsa.stattools import adfuller
```

```
adf_result=adfuller(monthly_sales)
```

```

print("ADF Statistic:",adf_result[0])
print("p-value:",adf_result[1])

#Train-Test Split

train=monthly_sales[:-6]
test=monthly_sales[-6:]

#Model Building(SARIMA)

from statsmodels.tsa.statespace.sarimax import SARIMAX //Time Series Forecasting Model

model=SARIMAX(

    train,

    order=(1,1,0),

    seasonal_order=(0,1,1,12))

model_fit=model.fit()

print(model_fit.summary())

#Prediction

sales_forecast=model_fit.forecast(steps=6)

sales_forecast.index=test.index

#Actual VS Forecast Plot

plt.figure(figsize=(8,5))

plt.scatter(train.index,train,label='Train')

plt.scatter(test.index,test,label='Actual Sales')

plt.scatter(sales_forecast.index,sales_forecast,label='Forecast Sales')

plt.legend()

plt.title("Sales Forecasting using SARIMA")

plt.xlabel("Date")

plt.ylabel("Sales")

plt.show()

#Model Evaluation

from sklearn.metrics import mean_absolute_error,mean_squared_error

import numpy as np//Numerical Computation

```

```
mae=mean_absolute_error(test,sales_forecast)
rmse=np.sqrt(mean_squared_error(test,sales_forecast))
print("MAE:",mae)
print("RMSE:",rmse)
#Future Sales Forecast(Next 12 Months)
future_sales=model_fit.forecast(steps=12)
plt.figure(figsize=(8,5))
plt.plot(monthly_sales,label="Historical Sales")
plt.plot(future_sales,label="Future Sales Forecast")
plt.legend()
plt.title("Next 12 Months Sales Forecast")
plt.show()
```

14. FUTURE SCOPE

The future scope of sales forecasting using historical data is broad and promising. Advanced machine learning and deep learning models such as LSTM, GRU, and Prophet can be implemented to enhance forecasting accuracy and capture complex sales patterns. The system can be improved by integrating real-time sales data, enabling dynamic and continuously updated forecasts. Incorporating external factors like seasonal events, promotions, holidays, economic trends, and weather conditions can further improve prediction reliability. Additionally, forecasting can be expanded to product-level, region-level, and store-level analysis to provide more detailed business insights. Interactive dashboards and visualization tools can also be developed to support better decision-making, while automated model retraining can help the system adapt effectively to changing market conditions.

GitHub Project Repository Screenshot

